PRODUCT REQUIREMENT DOCUMENT (PRD)

Project Name: UniGuide **Version:** 2.0 (sync with shipped state 2026-04-20) **Domain:** AI Systems & Agentic Workflow Automation (Domain 1) **Team:** Breaking Bank **Submission:** UMHackathon 2026 — Preliminary Round **Live deployment:** https://uniguide-blush.vercel.app **Repository:** https://github.com/ZenBen5173/uniguide

---

Table of Contents

1. Project Overview
2. Background & Business Objective
3. Product Purpose
4. System Functionalities
5. User Stories & Use Cases
6. Features Included (Scope Definition — MVP)
7. Features Not Included (Scope Control)
8. Assumptions & Constraints
9. Risks & Mitigations

---

1. Project Overview

Problem Statement

Every Malaysian university student is, sooner or later, blocked by the same kind of wall: an opaque multi-stage administrative procedure that no one has fully explained. Scholarship applications, postgraduate admissions, examination appeals, deferment of studies, FYP supervisor matching, EMGS visa renewals — each procedure spans 6 to 12 stages, involves 3 to 5 different offices, references regulations buried in PDFs, and has silent failure modes (a missed two-week appeal window, a private-clinic medical certificate that needed to be from a Government Hospital, a CGPA that's 0.20 short of an unwritten threshold, applying to MARA without realising it's Bumiputera-only). Today, students piece these procedures together from outdated forum threads and trial-and-error emails. Administrators on the receiving end then spend hours triaging incomplete, mis-routed submissions.

Target Domain

University administrative procedures at Universiti Malaya — guiding students through complex multi-step processes with adaptive decision-making, while pre-digesting their submissions for the staff who must approve them.

Proposed Solution Summary

UniGuide is an AI-driven workflow assistant where Z.AI's GLM acts as the central reasoning engine that reads the official UM SOP for whatever the student is trying to do, **emits the next step in the application at runtime — one step at a time — based on the SOP plus everything the student has answered so far**, walks the student through it adaptively (with citation chips showing which SOP section

drove each question), and on submission **generates a coordinator-side briefing** with extracted facts, flagged edge cases, and a recommended decision plus reasoning trace. The coordinator can preview, edit, and send the GLM-generated decision letter; the student receives it in real-time.
If the GLM component is removed, UniGuide cannot advance any application past step 1 — there is **no static workflow template** to fall back on. Every step is planned at runtime by GLM; every coordinator briefing, letter, and suggested comment is GLM-generated.

---

2. Background & Business Objective

Background of the Problem

University procedures at UM are documented in three principal corpora — the Bachelor's Degree Regulations (a 100+ page legal document), faculty-specific handbooks (a different one for every faculty and every cohort), and ad-hoc PDF forms scattered across `um.edu.my` subdomains (`hep.um.edu.my`, `fsktm.um.edu.my`, `ips.um.edu.my`, `study.um.edu.my`, `aasd.um.edu.my`). Three principal authorities own the decisions — the Faculty Dean (operational), the Deputy Vice-Chancellor Academic & International (escalation), and the Senate (final academic). Three principal student-facing portals expose the workflows — MAYA, SiswaMail, SPeCTRUM. None of these systems share a single source of truth.

Importance of Solving This Issue

- **Student welfare:** thousands of UM students per year miss critical deadlines (exam appeal: 2 weeks; EMGS visa renewal: 3 months in advance; corporate scholarships: narrow Feb–April window) or apply to schemes they don't qualify for, simply because the eligibility rules were never explained in one place.
- **Administrative cost:** every incomplete or mis-routed submission consumes minutes of staff time. At Yayasan UM scale (thousands of applications a year, many auto-rejected for ineligibility), this is hundreds of staff-hours that could be returned to actual review.
- **Compliance & audit:** the absence of a structured paper trail makes it difficult to audit consistency across faculties.

Strategic Fit / Impact

- Aligns with UM's digitalisation roadmap and the Ministry of Education's push toward standardised student services.
- Demonstrates **Z.AI GLM** as a load-bearing reasoning engine for a real-world Malaysian use case, not a generic chatbot wrapper.
- Inverts the conventional workflow-tool model: instead of admin-built static templates, GLM **emits the workflow at runtime** from the indexed SOP — strictly more adaptive than any template-based product.

---

3. Product Purpose

3.1 Main Goal

To provide every UM student with a personalised, adaptive AI guide for any administrative procedure — from procedure selection through filed paperwork — and to give every UM administrator a pre-digested submission briefing instead of a raw form to triage.

3.2 Intended Users (Target Audience)

Three role-distinct surfaces, all in the live app:
Primary user group — Students (`/student/portal`, `/student/applications/[id]`):

- Undergraduate students (scholarships, FYP, exam appeals, deferment)
- Postgraduate candidates (admission, supervisor matching, thesis submission)
- International students (EMGS visa renewal, additional MoE compliance)

Secondary user group — Coordinators / Staff (`/coordinator/inbox`, `/coordinator/applications/[id]`):

- Yayasan UM / Scholarship Office officers
- Faculty Postgraduate Committee members
- Faculty Deans and Deputy Deans (Academic)
- Examination & Graduation Section officers
- Academic Administration Division officers

Tertiary user group — Admins (`/admin`, `/admin/analytics`, `/admin/glm-traces`):

- IT or department leads who index SOPs, edit letter templates, set deadlines
- Compliance teams who audit the GLM reasoning trace

---

4. System Functionalities

4.1 Description

UniGuide is a stateful, adaptive workflow engine powered by Z.AI's GLM. The system presents a catalogue of available UM procedures to authenticated students. When a student starts an application, the engine fetches the relevant SOP chunks via vector retrieval from the knowledge base and asks GLM to **emit the first step** (form, file upload, free text, multiple choice, info card, etc.). The student responds; the engine persists the response, then asks GLM to **emit the next step** with the full history + SOP context. This continues until GLM determines the application is complete, at which point the student submits and the engine **generates a coordinator briefing**.
Coordinators receive the inbox sorted by AI urgency, click any row to see the briefing, and decide. On Approve / Reject, GLM **fills the procedure's letter template** against the application context; the coordinator can preview, edit, and send. A **hallucination check** flags any concrete fact in the letter (CGPA, name, faculty, programme, year) that doesn't match the application before the letter goes out.
Admins index SOPs (paste / URL / **upload PDF**), edit **letter templates**, set **deadlines**, and audit every GLM call on the **GLM Traces** page.

4.2 Key Functionalities (as shipped)

#	Functionality	Description
F1	**Procedure Catalogue + Sign-In Flow**	Landing explains the three roles. Login offers 3 demo tiles (Student / Coordinator / Admin) for instant access + email OTP for real users. New

#	Functionality	Description
		users are routed through onboarding (faculty, programme, year, CGPA, citizenship).
F2	**Adaptive Step Emission Engine**	When a student starts an application, GLM emits **one step at a time** with type, prompt text, config, and SOP citations. The engine inserts the step into `application_steps`, surfaces it to the student, and on response calls GLM again with the new history. Step types: form (multi-field, supports file fields), file_upload, text, select, multiselect, info, final_submit, coordinator_message.
F3	**Citation Surfacing**	Every AI-emitted step persists the SOP section names that informed it (e.g., `["Document Checklist", "Yayasan UM Pathway"]`). Rendered as clickable §-chips beneath the step prompt; click opens the SOP viewer modal pre-filtered to that section with the term highlighted.
F4	**Real-time Status Updates**	Supabase Realtime publication on `applications`, `application_steps`, `application_letters`, `application_messages`. Student application page subscribes and refreshes on coordinator decisions. Student portal subscribes for inbox-badge updates.
F5	**Coordinator Briefing**	On submission, GLM emits a structured briefing with extracted facts, flags (info / warn / block), recommendation (approve / reject / request_info), reasoning, and confidence. Persisted to `application_briefings`; surfaced as the front-and-centre card on the coordinator detail page.
F6	**Decide + Letter Preview & Edit**	Coordinator clicks **Preview & approve** / **Preview & reject** → modal calls a separate preview endpoint that runs the letter template through GLM **without committing the decision**. Coordinator can edit the letter inline. **Confirm & send** writes the decision + the edited letter; status flips to approved/rejected.
F7	**Hallucination Check on Letters**	Preview endpoint runs a regex-based check against the application context. Flags: unfilled `{{placeholder}}`s, mismatched CGPA, mismatched name in greeting, wrong

#	Functionality	Description
		year/faculty/programme. Issues surfaced as warn/block tints in the preview modal.
F8	**Undo Decision (5-min window)**	Coordinator can undo Approve/Reject within 5 minutes. Wipes the decision row + letter, reverts status to submitted. UI shows mm:ss countdown on the coordinator detail right rail.
F9	**Request More Info Loop**	Coordinator can request more info; comment becomes the seed for `nextStep(coordinatorRequest=…)`, which GLM uses to emit a new step into the student's flow with `emitted_by=coordinator`. Student sees a crimson "From coordinator" pill on the new step.
F10	**AI-Suggest Comment**	Three pills above the coordinator's comment textarea (Request info / Approve / Reject) call a separate GLM endpoint that drafts a tailored 1-3 sentence comment based on the briefing flags + decision intent. Pre-fills the textarea on click.
F11	**Bulk Coordinator Actions**	Inbox supports bulk-approve (auto-excludes flagged + low-confidence rows) and bulk-request-info (one shared message broadcast to N selected).
F12	**Coordinator Claim / Assignee**	`applications.assigned_to` (nullable). Inbox row shows assignee chip ("you" or first-name); detail page right rail has Claim / Take over / Release. **Mine** filter on inbox shows only claimed-by-me rows. Advisory only — RLS still allows any staff to act.
F13	**Internal Notes (staff-only)**	Separate `application_coordinator_notes` table, RLS-protected. Coordinator can compose, list, and delete-own notes on the detail page. Never shown to the student.
F14	**Persistent Message Thread**	`application_messages` table with author_role (student / coordinator). Both sides write into the thread; INSERTs are realtime-published so chat appears in the other tab in <1s. Distinct from formal coordinator_message steps.
F15	**Step Revise**	Student can click **Revise** on any completed step in a draft application. The endpoint clears that step's response + deletes all later steps; on next submit, GLM replans from the new answer.

#	Functionality	Description
F16	**Application Withdraw**	Student can withdraw an application from any pre-decision state (draft / submitted / under_review / more_info_requested). Audit row inserted into `application_decisions`.
F17	**Admin: SOP Upload (PDF / URL / Text)**	Three-step modal: paste text, paste URL, **or upload a PDF** (real `pdf-parse`). Image-only PDFs rejected with helpful message. SOP is chunked by H2 headings + word count, embedded into `procedure_sop_chunks`.
F18	**Admin: Letter Template Editor**	List, create, edit, delete acceptance / rejection / request_info templates per procedure. Sensible defaults pre-populated. Placeholders auto-detected from `{{...}}` regex on save.
F19	**Admin: Deadline Editor**	Per procedure, set `deadline_date` + display label. Surfaces on student portal cards as "X days left" (auto-computed) or custom label, with tone shifts as cutoff approaches.
F20	**Admin: Analytics**	KPI strip (total / this-week vs last / pending / avg decision time) + by-procedure table + status mix bars. Aggregated server-side at request time.
F21	**Admin: GLM Trace Viewer**	Every Z.AI call logged to `glm_reasoning_trace` (endpoint, model, prompt hash, input summary, output, confidence, latency, tokens, cache hit). Listed on `/admin/glm-traces` with search + endpoint filter; click row to expand input/output JSON. **The transparency story.**
F22	**Real File Upload (Supabase Storage)**	`application-files` private bucket, RLS-protected. Path scheme: `{user_id}/{application_id}/{step_id}-{filename}`. Owner uploads + reads; staff reads all. Coordinator detail renders a paperclip "(view)" link that fetches a 60s signed URL via `/api/files/sign`.
F23	**Auto-Save (real)**	Student application drafts persist to `localStorage` per-step (debounced 300ms), hydrate on remount. Cleared on submit. Save & exit affordance returns student to portal without losing input.
F24	**Notification Bell**	Header bell polls `/api/notifications` every 45s. Unread count from `localStorage` last_read_at. Dropdown lists last 14 days of

#	Functionality	Description
		events: status changes, letter delivery, decisions, new submissions, info requests.
F25	**Letter Print / Save as PDF**	Each delivered letter has an "Open / Print →" link to a clean printable page (`/letters/[id]/print`) with UM letterhead, reference number, recipient block. Browser print → save as PDF.
F26	**SOP Viewer for Students**	Modal accessible from the student application right rail. Lists all indexed chunks of the procedure's SOP with full-text search, section-list footer, link to the original source. Same component opens via citation-chip click on AI-emitted steps.
F27	**Profile Editor**	`/settings/profile` lets either role update full name / faculty / programme / year / CGPA / citizenship after initial onboarding. Top-bar avatar links here.
F28	**Demo Mode + Auto-Fallback**	Banner at top of every page when `GLM_MOCK_MODE=true`. GLM client has auto-fallback: if a real Z.AI call errors AND a mock fixture matches the call, returns the fixture with logged error → demo never collapses.

4.3 AI Model & Prompt Design

4.3.1 Model Selection

Selected models: Z.AI GLM-4.6 for reasoning-heavy endpoints (`nextStep`, `generateBriefing`, `fillLetter`); GLM-4.5-flash for high-volume / lower-stakes calls (`estimateProgress`, `suggestComment`).
Justification:

- **Mandatory by hackathon rules.**
- **Long context window** — fits the full UM procedure SOP (24+ chunks for Scholarship) plus the student's accumulated history in a single call without aggressive chunking.
- **JSON mode** — every endpoint returns structured JSON validated by Zod schemas (`lib/glm/schemas.ts`).
- **Model tiering** holds the per-workflow token budget under target.

4.3.2 Prompting Strategy

Each GLM-powered endpoint has a dedicated versioned system prompt in `lib/glm/prompts/*.md`:

Endpoint	Strategy	File	Reason
nextStep	Few-shot, JSON-mode, glm-4.6	`prompts/next_step.md`	Must emit valid step type + config + citations every time
generateBriefing	Few-shot with sample briefings, JSON-mode	`prompts/brief.md`	Consistent format across submissions
fillLetter	Zero-shot with template + facts, JSON-mode	`prompts/fill_letter.md`	Letter is human-edited after — slightly looser
estimateProgress	Zero-shot, JSON-mode, flash	`prompts/estimate_progress.md`	Just a number, fast
suggestComment	Zero-shot, JSON-mode, flash	(inline system prompt, suggest-comment route)	Drafts a 1-3 sentence coordinator comment from briefing + intent

4.3.3 Context & Input Handling

- **Document parsing:** PDFs via `pdf-parse` (Node-only, declared as `serverExternalPackages` in `next.config.ts`). Image-only PDFs rejected explicitly — OCR pathway is roadmap.
- **Per-call SOP context:** for `nextStep`, all SOP chunks for the procedure are included (current procedures fit in context; will switch to vector-retrieval beyond a threshold).
- **History:** all completed steps with their responses are passed to `nextStep` so GLM has full context.

4.3.4 Fallback & Failure Behaviour

- **Schema violation:** Zod parse on every GLM output. Failure throws → caller returns 500 with the specific error.
- **Hallucinated regulation references:** scope is limited to the indexed SOP chunks; the citation chips link to actual indexed sections. Hallucination check on the letter compares mentioned facts against the application context.
- **GLM API timeout / error:** if a `mockFixture` is named for the call, `callGlm` automatically returns the fixture (with `console.error` log + `model: …-fallback` marker). Demo never collapses.
- **Mock mode:** `GLM_MOCK_MODE=true` env var (or absent `ZAI_API_KEY`) routes every call to fixtures. Visible demo-mode banner at the top of every page.
- **Application status guard:** decide endpoint refuses if status is already final; revise endpoint refuses if not draft; undo endpoint refuses past 5-minute window.

---

5. User Stories & Use Cases

5.1 User Stories

As an undergraduate student, I want to pick a UM procedure from a clear catalogue, then have the AI ask me only the questions that apply to my situation — instead of filling a 12-page form where 8 pages are irrelevant. I want to see which SOP section the AI is reading from, so I can audit "why is the AI asking this?" I want to save my draft and resume later, and to get a real-time notification when the coordinator decides.

As a Yayasan UM scholarship coordinator, I want each application to arrive with the CGPA verified, the income tier inferred from the parsed EPF/payslip, the Bumiputera status confirmed, and any CGPA shortfall flagged with the student's hardship justification surfaced — so that I can approve standard B40 cases in seconds. When I want to ask for more info, I want the AI to draft the message based on the flags. When I approve, I want to preview the letter, edit it, and have the system check it for hallucinated facts before it goes to the student.

As an admin, I want to upload an SOP PDF and have UniGuide chunk + index it, then write the matching letter templates, set the deadline, and watch students start applying — without writing any application code. I want to see every GLM call the system has made today, with cost and confidence, so I can audit the AI's behaviour.

5.2 Use Cases

UC-1 — Student starts and completes an application

10. Student lands on `/student/portal` (signed in via demo tile or OTP).
11. Picks a Live procedure from the Available services grid.
12. Engine creates the application row + calls `nextStep` → emits Step 1.
13. Student answers; submits step → engine calls `nextStep` again → emits Step 2 with full history.
14. Repeat until GLM emits a `final_submit` step.
15. Student reviews the summary, hits **Submit application**.
16. Engine generates a coordinator briefing; status flips to `submitted`.

UC-2 — Coordinator reviews and approves

17. Coordinator opens `/coordinator/inbox` → sees the new submission at the top (sorted by AI urgency).
18. Clicks the row → detail page loads with AI Briefing card front-and-centre.
19. (Optional) Clicks **Claim** so other coordinators don't collide.
20. Reads briefing + steps + uploaded files (paperclip "(view)" links).
21. Types a comment (or clicks **AI suggest: approve** to draft one).
22. Clicks **Preview & approve** → modal opens with GLM-generated letter.
23. Reviews hallucination check warnings (if any), edits the letter inline.
24. Clicks **Confirm & send** → status flips to `approved`, letter delivered, student tab updates instantly.

UC-3 — Coordinator requests more info

25. Same setup as UC-2 through step 5.
26. Coordinator types what's missing (or **AI suggest: request info**).
27. Clicks **Request more info** → engine calls `nextStep(coordinatorRequest=text)` → emits a new step into the student's flow with `emitted_by=coordinator`.
28. Status flips to `more_info_requested`; student gets the new step + crimson "From coordinator" pill.

UC-4 — Student withdraws / revises

- **Revise:** Student clicks **Revise** on any completed step in a draft. Engine clears that step + deletes all later steps. On next submit, GLM replans from the new answer.
- **Withdraw:** Student clicks **Withdraw application** in the footer. Status flips to `withdrawn`; audit row inserted; files + draft retained for audit.

UC-5 — Coordinator undoes a decision within 5 minutes

29. Coordinator approves an application by mistake.
30. Right rail shows "Undo window · mm:ss left".
31. Clicks **Undo this decision** within 5 minutes → letter wiped, decision row deleted, status reverts to `submitted`. Application returns to the queue.

UC-6 — Admin indexes a new procedure

32. Admin opens `/admin` → clicks **+ New procedure** → 3-step modal.
33. Step A: chooses input mode (paste text / paste URL / upload PDF).
34. Step B: enters procedure id (auto-slugified) + name + description; pastes text or uploads PDF.
35. Step C: confirmation. Clicks **Add letter templates →** → procedure detail page.
36. Sets deadline (date + label).
37. Adds acceptance / rejection / request_info templates (sensible defaults pre-loaded; `{{placeholders}}` auto-detected).
38. Procedure now appears as Live on the student portal.

UC-7 — Admin audits AI behaviour

39. Admin opens `/admin/glm-traces`.
40. Sees KPI strip (total calls / avg latency / tokens / cache hit rate).
41. Filters by endpoint (e.g., only `nextStep`) or searches by workflow id.
42. Clicks any row → expands input + output JSON for that GLM call.

5.3 Use Case Diagram (textual)

```
            ┌──────────────────────────┐
            │        Student           │
            └──────────────────────────┘
                  │          │
                UC-1       UC-4
                  │          │
                  ▼          │
        ┌──────────────────────┐
        │ GLM Service Layer    │ ──── auto-trace ────► /admin/glm-traces (UC-7)
        │ nextStep, brief,     │
        │ fillLetter,          │
        │ suggestComment,      │
        │ estimateProgress     │
        └──────────────────────┘
                  │ UC-2 / UC-3 / UC-5
                  ▼
        ┌──────────────────────┐
        │   Coordinator        │
        └──────────────────────┘

        Admin ──► UC-6 (catalogue + SOPs + templates + deadlines)
```

---

6. Features Included (Scope Definition — MVP)

Live demo procedure — Scholarship & Financial Aid Application (Yayasan UM pathway) — implemented end-to-end:

- All 28 functionalities (F1–F28) above are wired and demonstrable
- Pre-seeded demo: 5 sample applications in varied states (mid-flow draft, high-confidence approve, low-confidence + flagged, approved, rejected) so coordinator inbox + analytics + GLM traces all populate immediately

Procedure catalogue: 6 procedures listed (Scholarship, FYP, Deferment, Exam Appeal, Postgrad Admission, EMGS Visa). Scholarship is Live; the other 5 surface with "Coming soon" badges so the catalogue communicates scope without misleading.

UI scope: three role surfaces, all responsive on student-facing pages (login, portal, application). Coordinator + admin desktop-optimised with horizontal scroll fallback on tablet/phone.

Demo accounts (zero-credential):

- `demo-student@uniguide.local`
- `demo-coordinator@uniguide.local`
- `demo-admin@uniguide.local`

---

7. Features Not Included (Scope Control — Cut for MVP)

- **Email delivery via SMTP.** Letters are generated and delivered in-app (with print-as-PDF), but not emailed. Listed as roadmap.
- **Server-side PDF generation.** Letter print uses the browser's print-to-PDF — sharper output via library is roadmap.
- **OCR for scanned PDFs.** Current `pdf-parse` rejects image-only PDFs explicitly. Tesseract.js / cloud OCR is roadmap.
- **Bahasa Melayu UI.** English only.
- **Voice intake.** Text only.
- **Real MAYA / SiswaMail / SPeCTRUM / EMGS API integrations.** Mocked — no third-party callout.
- **Payment processing.** Exam-appeal fees, EMGS fees mentioned in workflow but no gateway.
- **Multi-tenant isolation.** Single UM workspace.
- **Mobile native apps.** Web only (responsive on student surfaces).
- **AI-driven workflow re-planning beyond turn-by-turn step emission.** No upfront workflow JSON; we deliberately chose step-at-a-time emission because it's strictly more adaptive.
- **Chat-style intent intake.** The product opens with a procedure catalogue + structured onboarding — judges should know we considered chat intake and rejected it as adding ambiguity without value when the procedure list is small.

---

8. Assumptions & Constraints

LLM Cost Constraint

Token budget per average user session: target ≤ 60,000 tokens (input + output) for a complete Scholarship Application workflow. Estimated cost at GLM list pricing: under RM 0.50 per workflow.

Design decisions to hold the budget:

- **Model tiering** — `glm-4.5-flash` for `suggestComment` + `estimateProgress`; `glm-4.6` for `nextStep` + briefing + letter

- **Response-length caps** — JSON schemas constrain output (`prompt_text` max 1000 chars, etc.)
- **Tracing** — every call logged to `glm_reasoning_trace` so we can spot runaway costs immediately

Technical Constraints

- **GLM API only.** Per Judging Criteria, no fallback to other LLMs. Failure mode is fixture-based auto-fallback (tagged `model: …-fallback`) so demo continues, not silent substitution by another provider.
- **Document parsing.** PDFs via `pdf-parse`; image-only PDFs rejected with user-facing message.
- **Browser support.** Last 2 versions of Chrome, Firefox, Safari, Edge.
- **Data persistence.** Supabase Postgres + Storage in `ap-northeast-2` (Seoul). Vercel pinned to `sin1` (Singapore) for ~70ms RTT.

Performance Constraints

- **GLM call latency target:** p95 < 4 seconds for `nextStep` and briefing; p95 < 1.5 seconds for `suggestComment` and `estimateProgress`.
- **End-to-end "submit step → next step rendered":** target p95 < 5 seconds.
- **Realtime push:** target p95 < 1 second from `applications` row update to client UI refresh (Supabase Realtime channel).

User Input Constraints

- **Must be authenticated.** Public pages: landing + login + onboarding only.
- **File upload:** max 10 MB per file (storage bucket constraint), accepted MIME: PDF + JPG/PNG/WEBP + DOC/DOCX.
- **Profile prerequisites for student:** full name + faculty + programme + year + CGPA + citizenship — collected at onboarding, editable at `/settings/profile`.

---

9. Risks & Mitigations

#	Risk	Why It Matters	Current Mitigation (shipped)
R1	**GLM hallucinates a fact in the letter (wrong CGPA, wrong name, wrong programme).**	Letter goes to student with bad data; embarrasses the office.	Hallucination check (regex) on every preview-letter response. Compares mentioned values against application context. Block-tinted warnings rendered above the editable letter.
R2	**GLM emits a malformed step config (invalid type, missing field).**	UI cannot render the step; student is stuck.	Zod schema validation on every GLM output. Failure throws in the engine; caller returns 500 with specific error.
R3	**GLM emits a step with no progress-**	Application loops indefinitely.	`is_complete` flag in `NextStepOutput` — GLM signals when no more steps are needed.

#	Risk	Why It Matters	Current Mitigation (shipped)
	toward-completion.		Final_submit step type. Estimated total steps shown to student so loops are visible.
R4	**GLM API outage during live demo.**	Demo dies on stage.	Auto-fallback in `lib/glm/client.ts` — when a real call errors AND a `mockFixture` is named, returns the fixture with logged error. 14 fixtures cover the full scholarship flow + briefing + 4 letter types + 3 suggest-comment variants.
R5	**Coordinator approves wrong application by misclick.**	Letter goes out, student notified.	5-minute undo window with mm:ss countdown on the action panel. Clicking Undo wipes the decision row, deletes the letter, reverts status.
R6	**Two coordinators decide on the same application simultaneously.**	Duplicate effort or conflicting decisions.	`applications.assigned_to` advisory column + Claim / Take over / Release UI. Inbox surfaces who's claimed it.
R7	**Student loses draft due to closed tab.**	Frustration; dropped applications.	Real localStorage auto-save (debounced 300ms) per-step, hydrates on remount. Save & exit affordance.
R8	**RLS policy gap exposes another student's files.**	Privacy breach.	`application-files` storage bucket RLS: INSERT requires `(storage.foldername(name))[1] = auth.uid()::text`; SELECT requires owner OR staff/admin. `/api/files/sign` checks ownership before returning signed URL. Tested via demo accounts.
R9	**Coordinator sees an internal note that should have been a student-facing message** (or vice versa).	Privacy / professionalism breach.	Two distinct tables: `application_coordinator_notes` (RLS: staff-only) vs `application_messages` (RLS: owner + staff). UI labels notes "never seen by student" with lock icon.
R10	**Real-time updates fail silently.**	Student sees stale status, doesn't realise decision is in.	Polling fallback via notification bell every 45s. Status-change event also recorded in `glm_reasoning_trace` for audit.

#	Risk	Why It Matters	Current Mitigation (shipped)
R11	**Token cost overrun during hackathon judging.**	Burn through GLM credit before final round.	Mock mode default (`GLM_MOCK_MODE=true`) for shared deploys. Demo-mode banner makes this explicit. Real-mode roll-out is one env var flip.
R12	**Procedure SOP changes after indexing.**	Workflow guides students with stale information.	Re-upload via admin SOP modal wipes + re-indexes. `procedures.indexed_at` shown in admin detail + on SOP viewer ("indexed 18 Apr 2026").

Open Questions for Production

- Will UM faculty staff actually use a third-party tool, or does this need to live inside MAYA? (Roadmap: MAYA SSO + iframe embed.)
- What's the right legal posture for hosting student-uploaded documents outside UM infrastructure? (Roadmap: deploy to UM-controlled cloud or self-hosted Supabase.)
- Is there appetite for the system to auto-submit on the student's behalf, or should it always be advisory? (Current: always advisory — coordinator clicks send.)

End of PRD v2.0 — synced with shipped state as of 2026-04-20